

ClaudeFast Code Kit

Stop Guessing. Start Building with Confidence.

The battle-tested Claude Code setup with 17 specialized agents, 25+ expert-crafted skills built on Anthropic's official recommendations and best practices from top practitioners, 6 intelligent hooks, and a CLAUDE.md that just works. No more wondering if your configuration is holding you back.

Trusted by 200+ developers, founders, and solo builders

Part of the ClaudeFast Bundle. Looking to grow your product after you build it? The bundle also includes the ClaudeFast Growth Kit for marketing, content, and go-to-market workflows.

Does This Sound Familiar?

- **Every session feels like starting over.** Different code styles, missed requirements, inconsistent quality. You spend more time re-aligning Claude than actually building.
- **Context keeps vanishing.** You explain your project for the fifth time today because Claude "forgot" everything after 30 minutes. Token limits eating your budget while you repeat yourself.
- **Permission prompts destroy your flow.** "Allow file read? Allow file write? Allow bash command?" Click. Click. Click. Twenty interruptions per hour. Your momentum dies in a thousand permission dialogs.
- **You're building the same scaffolding every project.** Custom prompts, agent instructions, session management, task tracking... 29+ hours of setup before you write a single line of product code.
- **Compaction hits when you're finally making progress.** Context window full. "Summarizing conversation..." And there goes everything Claude just learned about your codebase.

The worst part? You know Claude is powerful. You've seen what's possible. But getting consistent, production-quality output feels like wrestling a different AI every session.

Meanwhile, other developers are shipping complete features in hours while you're still debugging prompts.

Here's What's Actually Happening

Claude Code isn't the problem. **Your setup is.**

Without proper architecture, Claude operates like a brilliant developer with amnesia. Every session, every task, every file read starts from scratch. No memory. No specialization. No intelligent routing.

The **Code Kit** fixes this with four breakthroughs:

1. **Context Min-Maxing** — 6x effective building time before compaction through intelligent context management
2. **17 Specialized Agents** — The right expert handles each task automatically, not a generalist guessing
3. **6 Intelligent Hooks** — Skill adherence, auto-permissions, context recovery, formatting, code quality, and library sync
4. **Team Orchestration** — /team-plan + /build pipeline for complex multi-phase work with automatic planning and execution

Drop one folder into your project. Start prompting. That's it.

What Builders Are Shipping

"Dropped it into my Claude Code folder, ran one command, and I was building features 10 minutes later. No more second-guessing if my setup is 'right'. It just works."

deeps.dev, Solopreneur

"Went from idea to live MVP in 2 weeks flat. The Infra skill handled my entire VPS deployment. Something that used to eat an entire weekend."

Federico Fan, Founder

"My codebase finally looks like a senior engineer wrote it. The skills enforce patterns I didn't even know I needed. Worth it for that alone."

Vel Mariappan, Startup Founder

Who This Is For

The Code Kit is built for you if:

- You're a **solo developer** who wants enterprise-quality code without hiring a team
- You're a **non-technical founder** building your first SaaS product
- You're an **agency owner** who needs to ship client projects faster
- You've **tried Claude Code** but got frustrated with context loss and inconsistent output
- You want to **ship production apps**, not just prototypes and demos

Who This Is NOT For

- **Enterprise teams with 10+ developers** — You probably need custom tooling.
- **Anyone expecting "no-code" magic** — You'll still read code, make decisions, and guide the AI. This is a force multiplier, not a replacement for thinking.

Why This Works: Context Min-Maxing

The problem: Claude's 200K context window sounds huge until you realize your project files, conversation history, and instructions consume 80% of it. Then compaction hits and everything gets summarized into oblivion.

Our solution: Two complementary strategies working together:

- **Central AI conserves.** The main Claude instance delegates frequently, preserving its context for coordination, strategic decisions, and your conversation. It never gets bogged down reading files.
- **Sub-agents maximize.** Each specialist agent loads everything it needs, reads all relevant files, gathers examples. Their context windows are temporary anyway. Unused capacity means lower quality code and weaker output.

Result: 6x more effective building time before compaction. Sessions that last hours, not minutes.

6 Intelligent Hooks

Hooks are automated behaviors that fire at specific points in your workflow. They handle orchestration so you stay in flow.

Skill Activation Hook

Auto-appends skill recommendations to your prompts before Claude even sees them. Result: 100% skill adherence without relying on AI memory. The right context loads at the right time, every time.

Permission Hook

LLM-powered auto-approval with a 3-tier safety system. Safe operations get instant approval. Dangerous commands get blocked. Edge cases get analyzed. Zero interruptions without sacrificing security.

Context Recovery Hook

Token-based backup triggers fire at 50k tokens used, then every 10k after. Before compaction events, the hook automatically parses your conversation transcript, extracts key decisions, progress, and context, and saves a structured backup to `.claude/backups/`. Backups are numbered sequentially and timestamped. Old backups (>14 days) are automatically archived into consolidated summaries. It works entirely on its own with no manual intervention.

The hook also powers the **status line monitor**, which displays real-time context usage, API budget, and backup status directly in your Claude Code interface:

```
* Actualizing... (2m 35s · ↓ 3.6k tokens)

>

Opus 4.6 | 70k / 200k | 35% used 69,855 | 49% free 97,144 | think: On
current: ●○○○○○ 10% | weekly: ●○○○○○ 25% | extra: ○○○○○○ $0.00/$30.00
resets 3:00am (1h36m) | resets Fri, 3:00pm | resets apr 1
-> .claude/backups/38-backup-10th-Mar-2026-1-12am.md
```

Live status line: model info, context tokens used/total, percentage free, API budget (session + weekly), peak/off-peak indicator with countdown, reset times, and latest backup path.

Formatter Hook

Automatically runs Prettier on files after Write/Edit operations. Every file Claude touches comes out formatted consistently. No more style drift between agents.

Biome Validator Hook

Runs Biome lint check on JS/TS/JSX/TSX/JSON/CSS after file changes. Catches code quality issues immediately. Graceful fallback if Biome is not installed in your project.

Library Sync Hook

Auto-syncs edits to library-managed files back to your central library repository at the end of every Claude turn. Single synchronous push via a Stop hook -- no debounce, no detached worker. Catches both Claude's tool edits and manual IDE edits. Completely inert until you set up your own library -- no manifest file means the hook exits silently. Part of the optional Central Library system for managing your `.claude` folder across multiple projects.

No more clicking through 20 permission dialogs per hour. No more losing your train of thought to "Allow bash command?" No more context loss after compaction.

Your 17-Agent Development Team

Stop hoping a generalist AI figures out what you need. **Get the right specialist every time.**

Strategic Command

- **Master Orchestrator** — Your project manager specialist. When Central AI needs in-depth planning, it delegates to this agent for task breakdown, specialist assignment, and strategic coordination.
- **Growth Engineer** — Strategic growth planning, marketing campaigns, and go-to-market execution. Coordinates the Growth Kit ecosystem as a "Fractional CMO."

Core Development

- **Frontend Specialist** — React 19, Next.js 15, TypeScript, Tailwind. Responsive design, accessibility, performance.
- **Backend Engineer** — Type-safe APIs, server actions, authentication flows, multi-tenant architecture.
- **Supabase Specialist** — PostgreSQL, Row Level Security, real-time subscriptions, edge functions.
- **Quality Engineer** — Vitest, Playwright, proactive pattern enforcement. Prevention beats detection. Also serves as validation agent for team orchestration workflows.

Elite Special Forces

- **Security Auditor** — OWASP Top 10, RLS validation, threat modeling. Catches vulnerabilities before production.
- **Performance Optimizer** — Bundle optimization, query tuning, Core Web Vitals. Target: Lighthouse 90+.
- **Debugger Detective** — Root cause analysis, cross-system debugging, five whys methodology.

Domain Experts

- **iOS Expert** — SwiftUI, native iOS patterns, App Store optimization.
- **Flutter Expert** — Cross-platform mobile, native bridges, platform adaptations.
- **n8n Builder** — Workflow automation, 80-92% node reduction, MCP integration.
- **SEO Specialist** — Technical SEO, content optimization, SERP analysis.
- **Content Writer** — Documentation, marketing copy, user-facing content.
- **Deep Researcher** — External research, best practices, competitive analysis.
- **Session Librarian** — Documentation organization, knowledge preservation.
- **Code Simplifier** — Code clarity, refactoring, and maintainability optimization.
- **Visual Explainer** — HTML visualization generation for plans, diffs, architecture diagrams, and data tables.

You don't assign work. The Master Orchestrator routes each task to the right specialist automatically.

25+ Domain Skills (Loaded On-Demand)

Skills are expert knowledge bases distilled from Anthropic's official documentation, top engineering blogs, GitHub discussions, and community wisdom. Each skill loads only when needed via the Skill Activation Hook.

Development & Architecture

- **react** — 70 performance rules from Vercel Engineering across 8 priority-ranked categories, including Next.js Cache Components with Partial Prerendering
- **frontend-design** — Enhanced version of Anthropic's own recommended skill. 11 aesthetic direction systems, interaction pattern classification, design systems reference, CSS extraction toolkit, accessibility requirements, dashboard hierarchy.
- **postgres-best-practices** — 34 Supabase-sourced optimization rules across 8 categories
- **auth** — BetterAuth + Next.js App Router, OAuth patterns, production deployment
- **email-templates** — Email design system with component patterns
- **visual-explainer** — Self-contained HTML visualizations with Mermaid diagrams, CSS Grid, light/dark themes
- **codestats** — Code intelligence CLI with dependency graphs, hotspot scoring, dead code detection. Requires `pip install cf-codestats` (Python 3.10+).
- **mobile-design** — Apple HIG, Material Design 3, WCAG 2.2 across 8 platforms (350+ rules) plus 6 official Google Android development skills (AGP 9, XML-to-Compose, Navigation 3, R8 analyzer, Play Billing, edge-to-edge)

Infrastructure & Integrations

- **infra-ops** — VPS, SSH, Docker, Coolify, Nginx, security hardening
- **payment-processing** — Polar, Dodo Payments integrations, checkout flows, and webhooks
- **analytics** — DataFast, Umami, event tracking, privacy compliance
- **dev-browser** — Browser automation via Sawyer Hood's CLI (Rust + Node.js + QuickJS sandbox), persistent state

Content & Growth

- **growth-kit** — Complete non-dev toolkit: idea-to-product, research, marketing, SEO, CRO, pricing, paid ads, growth loops, media creation, web crawling
- **documentation-research** — Context7 API + skills registries for live library docs
- **fumadocs** — Sidebar organization, centralized meta system, content collections

Framework & Workflow

- **session-management** — 8 session types with auto-detection (Development, Debugging, Migration, Review, TDD, Research, Growth, Repo-Port)

- **sub-agent-invocation** — Constitutional delegation protocol with parallel/serial dispatch and repo-port context injection
- **git-commits** — Semantic commit conventions
- **codebase-navigation** — Project structure exploration
- **agentic-builder** — Create custom skills with eval loops, agents, and slash commands. 3-way router with dedicated builders.
- **claude-api** — Build, debug, and optimize Claude API / Anthropic SDK apps with prompt caching
- **archon** — Visual task management with Kanban UI and hierarchical knowledge search

The framework improves itself. When you discover workarounds or patterns, Claude prompts you to capture them as reusable skills. This is how ClaudeFast stays current — it's not a static download, it's a living system that evolves with you and the community.

Team Orchestration: /team-plan + /build

For simple work, just prompt naturally. Claude handles it directly or dispatches to the right specialist.

For more elaborate, multi-step work, use the `/team-plan` command with your prompt to trigger structured planning:

```
YOUR PROMPT IN CLAUDE CODE:

/team-plan Build a SaaS dashboard with user authentication, Stripe billing, and an admin panel for managing users
```

This generates a structured plan file in `.claude/tasks/` with task breakdowns, specialist assignments, dependencies, and acceptance criteria. Claude presents the plan and waits for your approval before executing anything.

Once you approve, follow up with one of two execution commands:

```
EXECUTE WITH ISOLATED SUB-AGENTS (STANDARD COST):

/build .claude/tasks/saas-dashboard-plan.md
```

```
EXECUTE WITH COLLABORATIVE AGENT TEAMS (2-4X COST, BETTER COORDINATION):

/team-build .claude/tasks/saas-dashboard-plan.md
```

Use <code>/build</code> when	Use <code>/team-build</code> when
Tasks are independent and isolated	Agents need to coordinate on shared interfaces
Research-heavy or focused work	Cross-domain integration (frontend + backend + DB)
Cost-sensitive (1x tokens)	Real-time collaboration needed (2-4x tokens)

Important: Agent Teams are not necessarily better in all cases. For focused, independent tasks, `/build` produces great results at lower cost. Reserve `/team-build` for genuinely complex work requiring significant coordination between agents.

To use `/team-build`, launch Claude with Agent Teams enabled. Start your session with the environment variable set:

```
POWERSHELL:

$env:CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS = "1"; claude
```

```
BASH / MACOS:

CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1 claude
```

The included `justfile` provides a shorthand for this. If you have `just` installed (`brew install just`, `scoop install just`, or `cargo install just`), you can simply run:

```
SHORTHAND (USING JUSTFILE):
```

```
just team
```

Both methods do the same thing: set `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1` for that session only.

The Difference the Code Kit Makes

✗ Without the Code Kit

You: "Build a user dashboard"

Claude: "Here's some generic React code..."

[20 minutes Later]

You: "Remember, we're using Supabase..."

[45 min - Context Limit. Summarizing...]

You: "Why did you change the auth approach?"

Claude: "What auth approach?"

Result: 3 hours of frustration, half-finished code

✓ With the Code Kit

You: "Build a user dashboard"

[Skill Activation Hook fires]

[Permission Hook handles approvals]

Agents activate:

- Master Orchestrator breaks into 6 tasks
- Supabase Specialist sets up auth with RLS
- Backend Engineer creates server actions
- Frontend Specialist builds responsive UI
- Quality Engineer adds test coverage

Result: Production-ready dashboard in 20 minutes

Same Claude. Same tokens. Completely different results.

Real Projects. Real Timelines.

"Build a Customer Portal"

Request: "Create a customer portal with subscription management"

Delivered:

- Secure authentication (OAuth + email)
- Stripe subscription integration
- Responsive dashboard UI
- Real-time updates via Supabase
- Automated test suite
- Production deployment ready

Time: 4 hours (vs. 4 weeks building alone)

"My App Is Slow"

Request: "My app is loading slowly, fix it"

Delivered:

- Performance audit identifying 12 bottlenecks
- Optimized database queries (eliminated N+1 problems)
- Lazy-loaded components reducing initial bundle 60%
- Image optimization with next/image
- CDN configuration for static assets

Result: 3x faster load times in 1 hour (vs. days of guessing)

What's Inside the Code Kit

```
your-project/
├─ .claude/
│   ├─ agents/           # 17 specialized AI experts
│   ├─ skills/           # 25+ domain knowledge bases
│   ├─ hooks/            # 6 intelligent automation hooks
│   ├─ commands/         # /team-plan, /build, /team-build, /view, /library
│   ├─ tasks/            # Living documentation & sessions
│   └─ settings.json     # Hook config, status line, permissions
├─ CLAUDE.md             # Mission control (v5.3)
├─ justfile              # Shorthand commands (optional)
└─ [your code]           # Production-ready from day one
```

Every file is purposeful. Every agent is specialized.

Start Building in Under 5 Minutes

1. Download the Code Kit (instant access after purchase)
 2. Extract to your project — Drop the `.claude` folder, `CLAUDE.md`, and `justfile`
 3. Install permission hook: `npm i -g @abdo-el-mobayad/claude-code-fast-permission-hook`
 4. Configure permission hook: `cf-approve install && cf-approve config`
 5. Activate hooks and status line — The included `.claude/settings.json` contains all hook configurations and the status line monitor. If you have no existing settings, it works as-is. Otherwise, merge the `"hooks"` and `"statusLine"` sections into your own.
 6. Install Just (optional, for Agent Teams) — `brew install just` / `scoop install just` / `cargo install just`
 7. Open Claude Code and start prompting
-

The Real Cost of Building Without the Code Kit

DIY Setup (What You're Doing Now):

- 3 hrs — Researching Claude Code best practices
- 2 hrs — Writing custom agent prompts
- 4 hrs — Testing context management strategies
- 6 hrs — Building session management from scratch
- 3 hrs — Figuring out task organization
- 2 days — Trial and error on agent coordination
- 5 hrs — Optimizing token usage
- 8 hrs — Creating workflow documentation
- ??? — Wondering if your setup is even optimal

Total: 29+ hours of setup before you build anything

At \$100/hour consulting rate, that's \$2,900 in lost time.

With the Code Kit:

- **30 seconds** — Extract folder to project
- **5 minutes** — Configure hooks
- **Immediately** — Start building with 17 specialists

Total: Under 5 minutes. Then you're shipping.

Technologies Covered

- **Frontend:** React 19, Next.js 15, TypeScript, Tailwind CSS
 - **Mobile:** SwiftUI (iOS), Flutter (cross-platform)
 - **Backend:** Node.js, Server Actions, REST/GraphQL
 - **Database:** PostgreSQL, Supabase, Prisma
 - **Testing:** Vitest, Playwright, React Testing Library
 - **Automation:** n8n workflows, custom MCP servers
 - **DevOps:** Vercel, Docker, GitHub Actions
 - **Payments:** Stripe, Polar, webhooks
 - **And 50+ more integrations...**
-

Common Questions

Q: Do I need coding experience?

No. Many Code Kit users started with zero coding knowledge. The Master Orchestrator breaks complex tasks into understandable steps, and Claude explains everything as it builds. You will learn to read code over time, but you don't need to start there.

Q: Is this just fancy prompts?

No. The Code Kit includes 17 specialized agents with distinct expertise, progressive skill disclosure, context isolation between agents, constitutional delegation protocols, intelligent task routing, and 6 hooks that automate orchestration. This is architecture, not prompts.

Q: What if I get stuck?

The living documentation means Claude remembers your project context across sessions, so debugging gets easier over time. Plus, the framework is self-documenting — every decision and pattern is captured automatically.

Q: What can I actually build?

Full-stack web apps, SaaS products, marketplaces, dashboards, APIs, mobile apps (iOS/Flutter), Chrome extensions, n8n workflows, and more. If it runs on modern web technology, the Code Kit can help you build it.

Q: How is this different from other Claude Code kits?

ClaudeFast launched alongside Claude Code itself — it's the oldest framework on the market and has been constantly evolving ever since. Every agent, skill, and workflow represents a consolidation of best practices from across the entire ecosystem: Anthropic's official blogs, top engineer posts, GitHub discussions, community wisdom. Other kits are snapshots. ClaudeFast is a living framework that grows with the community.

Q: What happens when Claude updates?

We update ClaudeFast to leverage new features. v5.3 is optimized for Opus 4.7 with 1M context window support. All updates included with your purchase.

Your AI Dev Team Is Ready

17 specialists. 25+ skills. 6 intelligent hooks. Team orchestration. Central library sync. Living documentation that writes itself.

Stop losing context. Stop clicking permission dialogs. Stop rebuilding scaffolding.

Start shipping production code in hours, not weeks.

Get the Code Kit and transform how you build.